Assignment 5 Strategy Pattern Report

Student: Timur Naumov
Group: SE-2537
Course: Software Design Patterns
Academic year: 2026-2027
Chosen option: B - Calculator

1. Introduction

The chosen domain is a calculator. Its algorithm family consists of addition, subtraction and multiplication of two numbers. Every algorithm accepts two `double` operands and returns one `double` result through `OperationStrategy.execute(a, b)`.

`Calculator` is the Context: it stores the selected strategy and delegates the calculation. `Main` is the Client: it selects the strategy and replaces it at runtime. This separates the choice of an operation from its execution. [1, 2]

2. UML class diagram

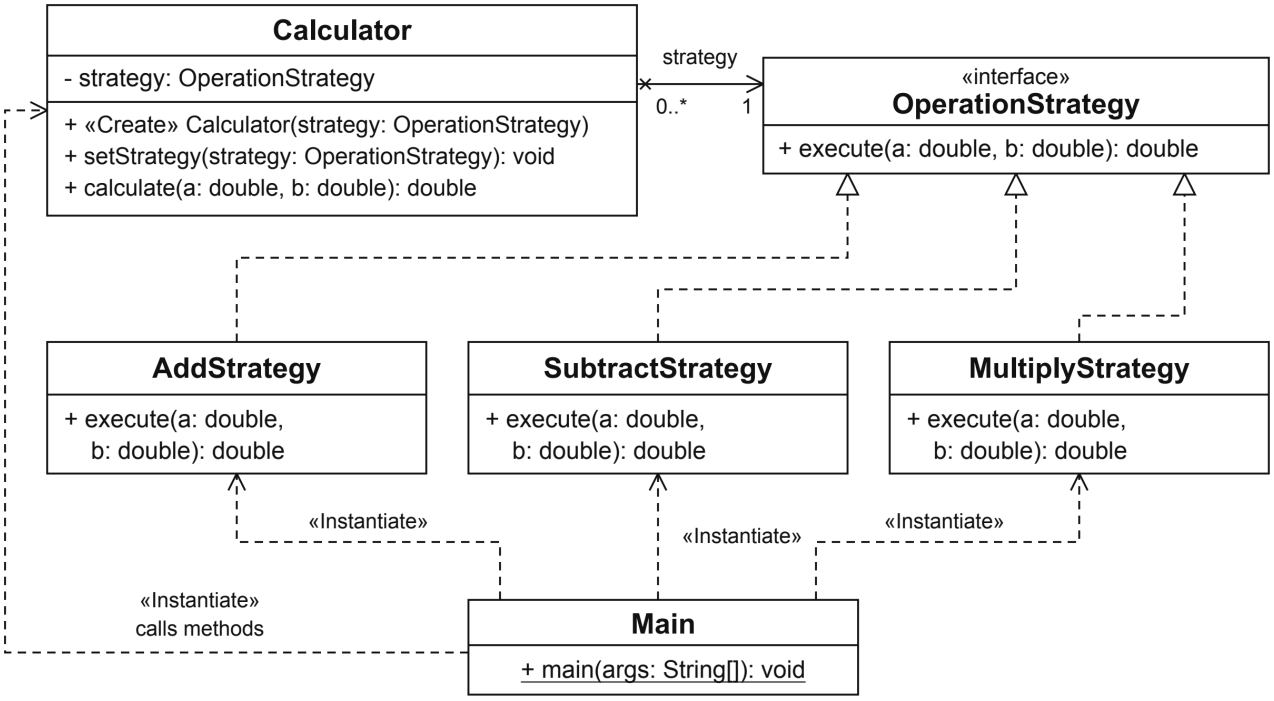


Figure 1. The six core application types in `src/`*. Tests and the optional division extension are outside this diagram's scope. Full-size SVG and editable PlantUML are included.*

UML notation and relationship directions

- **Interface realizations (3):** `AddStrategy`, `SubtractStrategy` and `MultiplyStrategy` each point to `OperationStrategy` with a dashed line and a hollow triangular arrowhead. This represents Java `implements`; the triangle points to the interface.
- **Navigable association (1):** `Calculator` points to `OperationStrategy` with a solid line and an open arrowhead. The small `x` at the Calculator end explicitly excludes reverse navigation: the strategies do not store a Calculator reference. The private field `strategy` and this association are two views of the same reference, not two separate fields.
- **Multiplicities:** `1` at the interface end means every fully constructed Calculator has exactly one selected strategy; the constructor and setter reject null. `0..*` at the Calculator end means one strategy object can be referenced by zero or more calculators. It does not mean that one calculator stores several strategies.
- **Creation dependencies (4):** dashed lines with open arrowheads point from `Main` to `Calculator`, `AddStrategy`, `SubtractStrategy` and `MultiplyStrategy`. The standard stereotype «`Instantiate`» records the constructor calls. `Main` also calls the Calculator methods; it stores the Calculator in a local variable, so no persistent association is modeled.
- **Visibility and members:** + means public and - means private. Compartments separate names, fields and operations. Underlined `main` is static; «`interface`» identifies the interface. «`Create`» on `Calculator(...)` marks the constructor. Its Java declaration has no written return type.

All relationship lines use the same normal width; thickness adds no meaning here. A lollipop circle is an alternative compact notation for a provided interface. The rectangular interface is used to show its operation. The association models a replaceable reference without whole-part semantics, so no aggregation diamond is needed. No composition diamond is used because exclusive lifetime ownership is absent and a strategy can be shared. [3, sections 7.7.4-7.7.5, 9.4.4, 10.4.4, 11.4.4, 11.5.4-11.5.5]

Responsibilities and execution flow

`Main` creates `new AddStrategy()` and passes it to a new `Calculator`. The constructor stores this object in the private `strategy` field. When the client calls `calculate(10, 5)`, the Calculator calls `strategy.execute(10, 5)`, which returns `15.0`. The Calculator returns this result to Main for printing.

Next, `Main` calls `setStrategy(new SubtractStrategy())` on the same Calculator; the next result is `5.0`. Replacing the strategy with `MultiplyStrategy` produces `50.0`. Switching back to addition produces `15.0` again. The Calculator object is never recreated, and no strategy knows about another strategy.

3. Clean Code principles

3.1 Replace conditional selection with polymorphism

A hypothetical conditional version couples the context to every operation:

```
// Before: illustration, not part of the submitted application.
if (operation.equals("+")) return a + b;
if (operation.equals("-")) return a - b;
if (operation.equals("*")) return a * b;
```

The submitted `Calculator.calculate` delegates instead:

```
public double calculate(double a, double b) {
    return strategy.execute(a, b);
}
```

The method has no `if`, `switch`, `instanceof` or concrete-strategy checks. Java dispatches the call to the selected implementation. Adding an operation does not add a branch to the context. [2]

3.2 Use meaningful names

```
calculator.setStrategy(new SubtractStrategy());
System.out.println("10 - 5 = " + calculator.calculate(a, b));
```

`SubtractStrategy` names the algorithm, `setStrategy` names replacement, and `calculate` names execution. These names make the client understandable without comments explaining numbered modes such as `operation = 2`.

3.3 Keep one responsibility per strategy

```
public class MultiplyStrategy implements OperationStrategy {
    @Override
    public double execute(double a, double b) {
        return a * b;
    }
}
```

This class only multiplies two operands. It does not read input, print results, choose another strategy or manage the Calculator. Presentation stays in Main; reference management stays in Calculator. [4]

3.4 Depend on an abstraction

```
private OperationStrategy strategy;

public void setStrategy(OperationStrategy strategy) {
    this.strategy = Objects.requireNonNull(strategy, "Strategy must not be null");
}
```

The field and setter use the interface. Calculator does not import or construct `AddStrategy`, `SubtractStrategy` or `MultiplyStrategy`. A null check protects the reference invariant; it does not choose an algorithm. The constructor enforces the same invariant.

3.5 Extend by adding a new strategy

The independent extension in `examples/DivideStrategy.java` adds division:

```
public class DivideStrategy implements OperationStrategy {
    @Override
    public double execute(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return a / b;
    }
}
```

A separate client, `examples/ExtensionDemo.java`, selects it:

```
calculator.setStrategy(new DivideStrategy());
System.out.println("10 / 5 = " + calculator.calculate(10, 5));
```

No file in `src/` needs modification. Only the new algorithm and new client are added; the stable context and interface are reused. This demonstrates the Open/Closed Principle. The divisor check validates input inside one algorithm. [2, 4]

Verification: 12 core checks and 5 extension checks pass. They include runtime switching and switching back, delegation to an unknown strategy, operand order, fractional and negative values, null rejection and division by zero. Compilation and execution both use Temurin JDK 17.0.21; `javac --release 17 -Xlint:all` reports no warnings.

4. Strategy vs. State

Strategy fits because the client deliberately chooses one of several interchangeable arithmetic algorithms. Replacing addition with subtraction changes how a calculation is performed, without moving the Calculator through a lifecycle. State addresses behavior that depends on an object's internal state and transition rules, such as Draft becoming Published. State objects may initiate transitions. Here, strategy objects do not initiate transitions or reference each other; Main selects them. Similar class structures therefore express different intentions. [2, 5]

5. Conclusion

The Strategy pattern keeps Calculator small and independent of concrete operations. A new algorithm can be added and selected without changing the context or the existing strategies. The cost is more classes and explicit client configuration: six core types are used for three simple operations. This is useful when algorithm variants need to grow or change independently, although it adds structure compared with a small fixed calculator. [2]

6. Link to the GitHub repository

https://github.com/tnaumovv/assignment5-strategy-pattern

References

[1] Astana IT University. *Assignment 5 Strategy Pattern*, Software Design Patterns, 2026-2027. Supplied assignment PDF.

[2] Refactoring.Guru. *Strategy*. https://refactoring.guru/design-patterns/strategy

[3] Object Management Group. *Unified Modeling Language 2.5.1*, formal/2017-12-05. https://www.omg.org/spec/UML/2.5.1/PDF

[4] Martin, R. C. *Clean Code*, Chapter 10. Prentice Hall, 2008. Course reading reference.

[5] Refactoring.Guru. *State*. https://refactoring.guru/design-patterns/state

Preparation assistance: OpenAI ChatGPT/Codex assisted with the example implementation, report and UML. The code was compiled, the examples were executed, and the diagram was checked against the source.